MANUAL DEL PROYECTO

Red de Historiales Clínicos

Qué hace cada archivo, cómo se comunican las piezas y dónde se conecta la base de datos

Universidad Rafael Landívar
Bases de Datos II · Proyecto final
Noviembre de 2026 · cuatro instituciones integradas

Contenido

1. Cómo leer este manual . . . 5

Conceptos básicos . . . 5

2. Qué es el proyecto . . . 6

3. Mapa general . . . 7

Las carpetas principales . . . 7

Quién habla con quién, según cómo se arranque . . . 8

4. Dónde se conecta la base de datos . . . 9

El archivo de conexión . . . 9

Qué se configura en .env . . . 9

Qué ocurre al arrancar el servidor . . . 9

Cómo llega la conexión hasta las consultas . . . 10

Todos los puntos que abren una conexión . . . 10

Las bases de las demás instituciones . . . 11

5. Cómo está guardada la información . . . 12

Índices . . . 12

Validación en dos niveles . . . 13

Cómo no se pierde la auditoría . . . 13

6. Cómo viaja una petición . . . 13

Ejemplo: un médico guarda una consulta nueva . . . 13

Cómo se inicia sesión . . . 14

Un solo formulario para las cuatro instituciones . . . 14

Qué pasa cuando algo falla . . . 15

7. Cómo está organizado el código . . . 15

Backend: arquitectura hexagonal . . . 15

Frontend: una plataforma y módulos enchufables . . . 16

8. Mapa de rutas: de la pantalla a la colección . . . 17

9. Roles y permisos . . . 18

10. Dónde está cada control de seguridad . . . 19

11. Catálogo de archivos . . . 20
Raíz del repositorio . . . 20
docs/ — documentación común . . . 20
apps/web/ — configuración de la aplicación web . . . 20
apps/web/src/ — arranque de React y estilos . . . 21
plataforma/ — utilidades y cliente HTTP . . . 21
plataforma/layout/ — estructura visual común . . . 21
plataforma/modulos/ — cómo se conectan las instituciones . . . 22
plataforma/paginas/, portal/ y sesion/ . . . 22
plataforma/ui/ — componentes reutilizables . . . 22
plataforma/vista-previa/ — solo en desarrollo . . . 23
Pediatría · backend: arranque y conexión . . . 23
Pediatría · backend: capa HTTP . . . 24
Pediatría · backend: dominio, casos de uso y puertos . . . 24
Pediatría · backend: adaptador MongoDB . . . 25
Pediatría · frontend: configuración del módulo . . . 25
Pediatría · frontend: páginas . . . 25
Pediatría · frontend: componentes . . . 26
Pediatría · scripts de operación . . . 26
Pediatría · documentación . . . 27
Pediatría · pruebas . . . 27
Hospital General (MySQL) . . . 28
Clínica Odontológica (Cloud Firestore) . . . 30
Laboratorio Clínico (PostgreSQL) . . . 32
tests/ — pruebas de la plataforma común . . . 35

12. Qué archivo tocar para cada cambio . . . 36

13. Instalación y comandos . . . 36
Variables de entorno . . . 37

14. Cómo conectar otra institución . . . 38

15. Problemas frecuentes . . . 38

16. Observaciones del análisis . . . 39
Pendientes conocidos . . . 39
Documentación desactualizada . . . 39
Límites aceptados del diseño . . . 39

Para seguir leyendo 39

1. Cómo leer este manual

Este manual explica el proyecto **Red de Historiales Clínicos** para alguien que abre el repositorio por primera vez: qué hace cada archivo, cómo se comunican las piezas y dónde se conecta la base de datos.

Si quiere saber…	Vaya a
Dónde se conecta la base de datos	Sección 4
Cómo está guardada la información	Sección 5
Qué pasa cuando alguien pulsa «Guardar»	Sección 6
Qué pantalla llama a qué ruta y qué colección toca	Sección 8
Qué hace un archivo concreto	Sección 11
Qué archivo tocar para hacer un cambio	Sección 12
Cómo instalarlo o por qué falla	Secciones 13 y 15

Para abreviar, `pediatria/backend/db.js` significa `instituciones/pediatria/backend/db.js`, y `plataforma/` significa `apps/web/src/plataforma/`.

> **Estado que describe**
> Rama `main` con las cuatro instituciones integradas, 8 de noviembre de 2026. El contenido se obtuvo leyendo el código. Las cantidades de pruebas y los estados de la base real se citan de la documentación del repositorio, porque para este manual no se ejecutó la aplicación ni se consultó la base.

Conceptos básicos

Término	En este proyecto significa
API	El servidor al que la interfaz le pide datos o le ordena guardar. Aquí usa Express.
Backend / frontend	Servidor (Node.js) / lo que corre en el navegador (React).
JWT o token	Credencial firmada que dice quién es el usuario, su rol, su institución y cuándo vence.
Driver	Librería `mongodb` que permite a Node.js hablar con la base.
Colección / documento	En MongoDB, equivalen más o menos a tabla / fila, pero un documento puede contener listas y objetos.
Embebido / referenciado	Guardado dentro del documento / guardado aparte y enlazado por un id.
Índice	Estructura que acelera búsquedas; si es único, impide valores repetidos.
Validador	Regla `$jsonSchema` que la base aplica por su cuenta antes de aceptar un documento.
Puerto / adaptador	Puerto: lista de operaciones de datos que el sistema necesita. Adaptador: su implementación para un motor concreto.
Standalone / replica set	MongoDB de un solo servidor (sin transacciones entre documentos) / grupo que sí las admite.
Outbox	Guardar el evento de auditoría dentro del mismo documento y entregarlo después, para no perderlo.

2. Qué es el proyecto

Es el proyecto final de Bases de Datos II de la Universidad Rafael Landívar: una plataforma de historiales clínicos compartida por cuatro instituciones. La idea central es **una sola aplicación web** y, detrás, **un backend y un motor de base de datos propios para cada institución**. El token del usuario decide a qué institución se dirige cada operación.

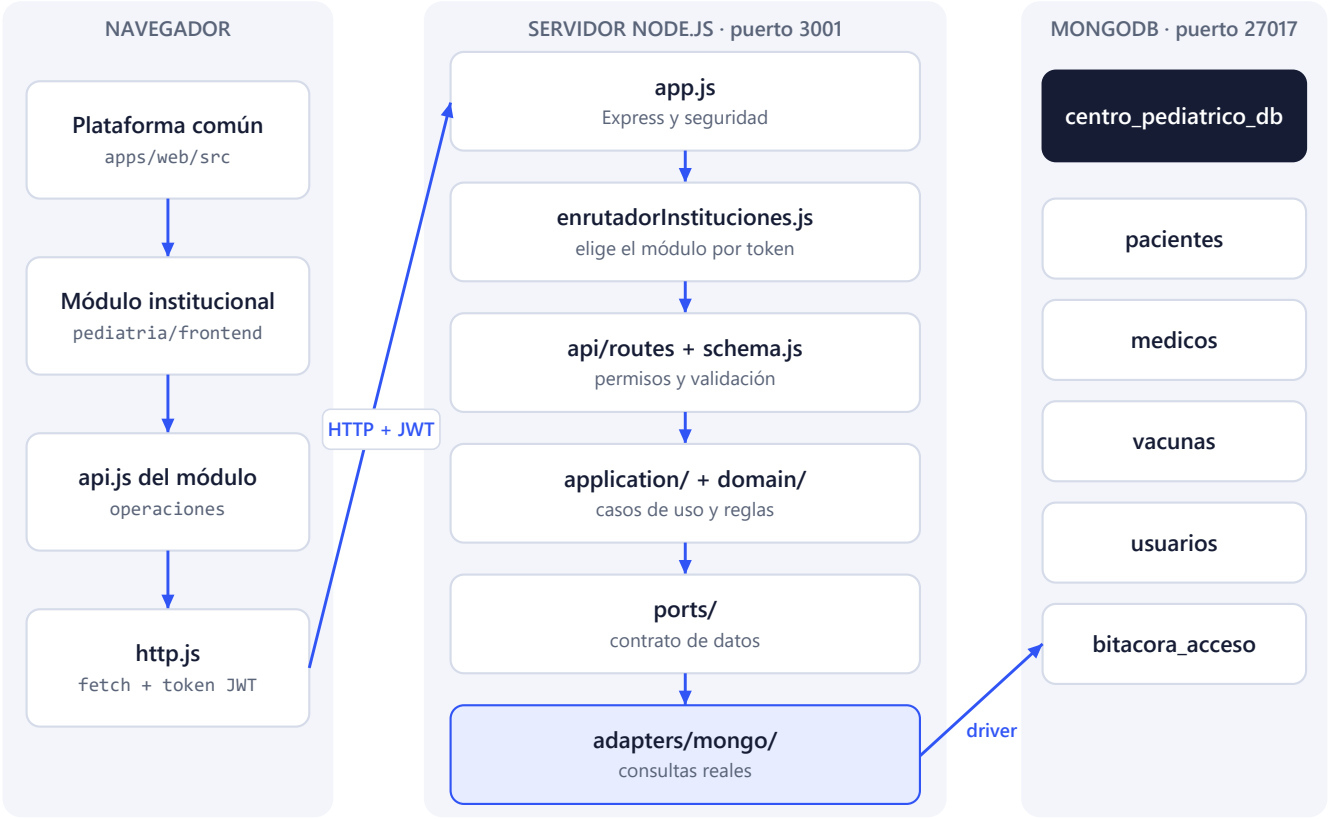
Institución	Carpeta	Identificador	Responsable	Estado actual
Centro Médico Pediátrico	`pediatria`	`centro_pediatrico`	Abel	**Implementado:** MongoDB, API, interfaz, pruebas y documentación
Hospital General	`hospital`	`hospital_general`	Ely	**Implementado:** MySQL, API e interfaz
Clínica Odontológica	`odontologia`	`clinica_odontologica`	Eddy	**Implementado:** Cloud Firestore, API, interfaz y pruebas
Laboratorio Clínico	`laboratorio`	`laboratorio_clinico`	Eddy	**Implementado:** PostgreSQL, API, interfaz y pruebas

Importante
Cada institución tiene **su propia base de datos**: MongoDB, MySQL, PostgreSQL o Cloud Firestore. El servidor es uno solo y, según la institución de la sesión, dirige cada petición a su módulo. Las secciones 4 a 10 explican con detalle Pediatría, que sirve de modelo; la sección 4 resume cómo se conecta cada una de las demás. Todos los datos son ficticios y el sistema no genera indicaciones clínicas.

Parte	Tecnología
Interfaz	React 18, React Router 7, Vite 8
Servidor	Node.js 20.19 o superior, Express 4, Zod (validación), JSON Web Tokens
Base de datos	MongoDB 9.0 con el driver oficial `mongodb` 6
Pruebas	`node:test` y `playwright-core`

3. Mapa general

El sistema tiene tres piezas. El navegador nunca habla directamente con la base: siempre pasa por el servidor.

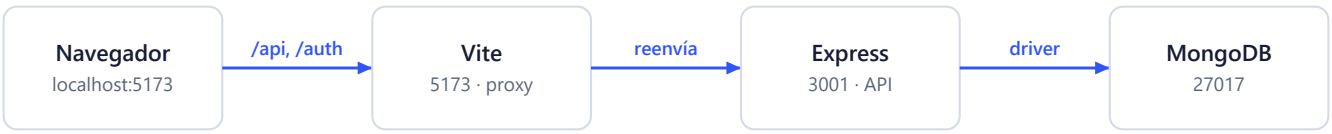


Las carpetas principales

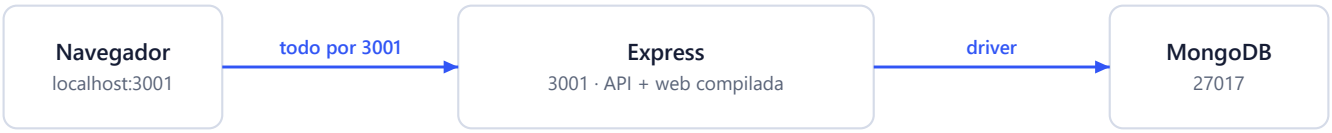
Carpeta	Contenido
`apps/web/`	La única aplicación web. `src/plataforma/` tiene lo común: sesión, permisos, cliente HTTP, menú y componentes.
`instituciones/<x>/frontend/`	Las pantallas y operaciones de cada institución. Se cargan dentro de `apps/web`.
`instituciones/pediatria/backend/`	La API y la conexión a MongoDB. Hoy también actúa como **servidor común** de toda la plataforma.
`instituciones/pediatria/scripts/`	Índices, validadores, diagnóstico, respaldo y restauración.
`instituciones/pediatria/tests/` y `tests/`	Pruebas de Pediatría y de la plataforma común.
`docs/`	Guía del equipo, sistema visual y este manual.

Quién habla con quién, según cómo se arranque

EN DESARROLLO · npm run dev + npm run web:dev



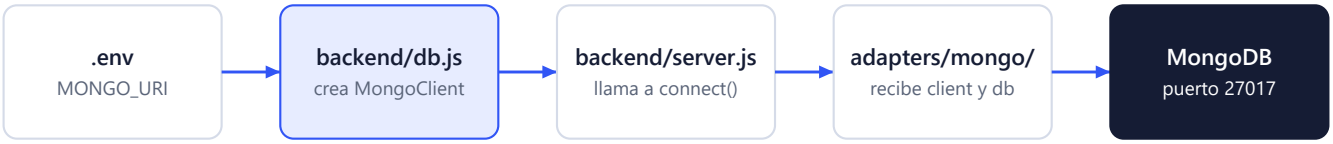
COMPILADO · npm run build + npm start



En desarrollo, el navegador solo conoce el puerto 5173 y `vite.config.js` reenvía las llamadas al 3001. Después de compilar, Express sirve también la aplicación, así que todo ocurre en el 3001.

4. Dónde se conecta la base de datos

La respuesta corta
Cada institución tiene su propio archivo de conexión; la tabla del final de esta sección los resume. Pediatría se conecta a MongoDB en un solo archivo: `instituciones/pediatria/backend/db.js`. La dirección y el nombre de la base no están escritos en el código: se leen del archivo `.env` en la raíz del proyecto.



El archivo de conexión

```
const { MongoClient } = require('mongodb');
const client = new MongoClient(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017', {
  appName: 'centro-pediatrico-api',
  serverSelectionTimeoutMS: 8000,   // espera hasta 8 segundos por el servidor
  maxPoolSize: 20                   // hasta 20 conexiones simultáneas
});
async function connect() {
  await client.connect();
  return client.db(process.env.MONGO_DB || 'centro_pediatrico_db');
}
module.exports = { connect, client };
```

Qué se configura en .env

```
MONGO_URI=mongodb://127.0.0.1:27017
# o, con usuario:
# MONGO_URI=mongodb://<usuario>:<contraseña>@127.0.0.1:27017/?authSource=admin
MONGO_DB=centro_pediatrico_db
```

Para apuntar a otra base solo se edita `.env`; el código no cambia. El archivo `.env` no se sube a Git, y cada integrante crea el suyo a partir de `.env.example`.

Qué ocurre al arrancar el servidor

Todo está en la función `start()` de `pediatria/backend/server.js`, en este orden:

#	Paso	Dónde	Si falla
1	Carga `.env`. Debe ir primero, porque `db.js` lee las variables en cuanto se importa	`server.js`	Se usan los valores por defecto
2	Comprueba `JWT_SECRET` (32 caracteres o más)	`security.js`	El servidor no arranca
3	Abre la conexión y hace `ping`	`db.js`	El servidor no arranca (`ECONNREFUSED`)

#	Paso	Dónde	Si falla
4	Crea los índices que falten	`adapters/mongo/index.js`	Si falla uno único, no arranca
5	Crea los repositorios con `client` y `db`	`crearRepositoriosMongo`	—
6	Detecta si hay transacciones y qué permite el validador	`unidadDeTrabajo.js`, `index.js`	—
7	Entrega auditoría pendiente; repite cada 60 s	`reenviarPendientes`	Se registra el error y sigue
8	Une casos de uso y Express, y escucha en `PORT`	`crearServicios`, `crearApp`	Mensaje si el puerto está ocupado

Cómo llega la conexión hasta las consultas

`server.js` crea los repositorios una sola vez y se los pasa a los casos de uso. Por eso, los casos de uso guardan y leen datos sin saber que existe MongoDB:

```
const db    = await connect();                    // db.js
const repos = crearRepositoriosMongo(client, db);  // adapters/mongo/index.js
const app   = crearApp(crearServicios(repos));     // application/ y app.js
```

Todos los puntos que abren una conexión

Además de la API, algunas herramientas abren su propia conexión. Esta es la lista completa de usos de `new MongoClient` en el proyecto:

Archivo	Para qué	A qué base	Permiso
`backend/db.js`	La API, `npm run db:indices` y `npm run db:crear-accesos`	`MONGO_URI` + `MONGO_DB`	Lectura y escritura
`scripts/validadores.js`	Verificar, aplicar o revertir validadores	`MONGO_URI`; para cambios, `MONGO_ADMIN_URI` o `--usuario`	Modificar requiere `dbAdmin`
`scripts/respaldo/diagnostico.js`	Informe del estado de la base	`MONGO_URI`	Solo lectura
`scripts/respaldo/respaldar.js`	Respaldo con `mongodump`	`MONGO_URI`	Solo lectura
`scripts/respaldo/restaurar-aislado.js`	Comprobar un respaldo	`mongod` temporal; opcionalmente lee el origen	Nunca escribe en la original
`tests/soporte/mongodTemporal.js` y pruebas de integración/E2E	Pruebas automáticas	`mongod` desechable en puerto libre	Nunca toca la base real

Regla para el equipo
Las consultas de la aplicación están solamente en `adapters/mongo/`. Ni las rutas, ni los casos de uso, ni el navegador deben usar el driver. Cada institución futura tendrá su propia conexión, con variables con prefijo (`HOSPITAL_DB_URL`…), sin reutilizar la de MongoDB.

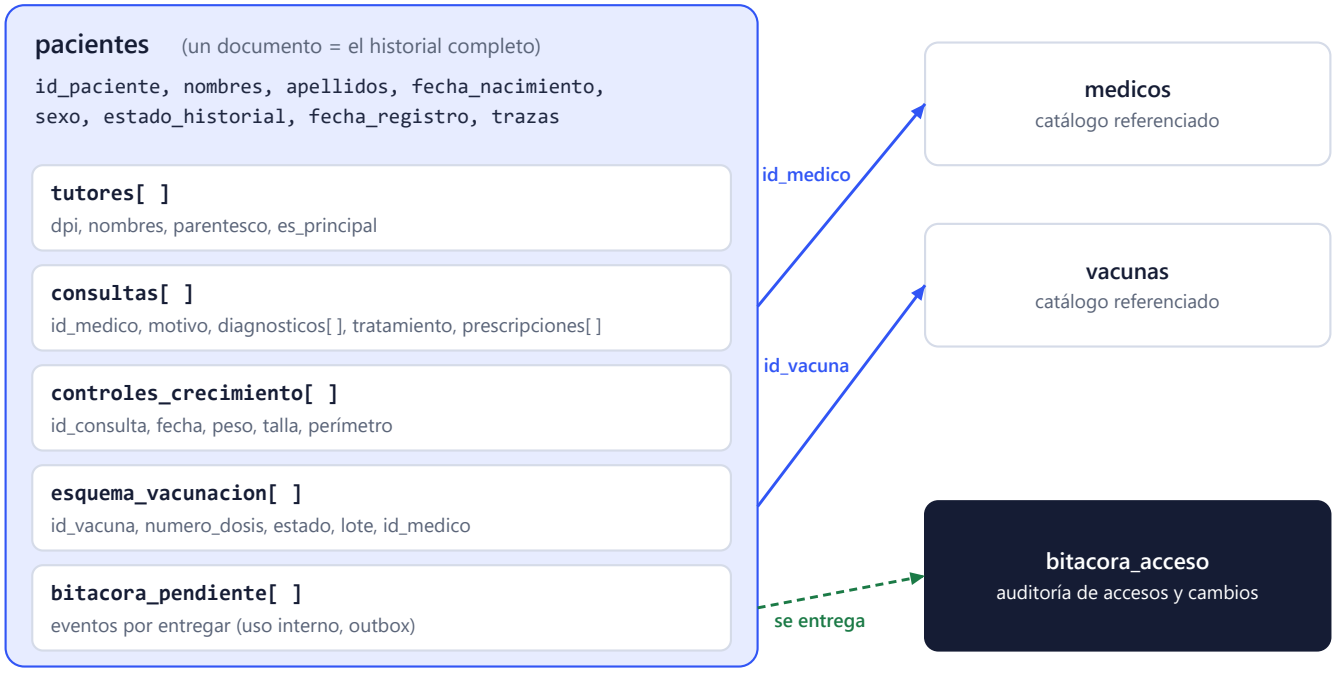
Las bases de las demás instituciones

El servidor común arranca desde `pediatria/backend/server.js` y monta cada institución **solo si su variable de conexión existe en `.env`**. Si su base no responde, esa institución contesta `503` y las demás siguen funcionando.

Institución	Motor	Archivo de conexión	Variable en `.env`
Pediatría	MongoDB	`pediatria/backend/db.js`	`MONGO_URI`, `MONGO_DB`
Hospital General	MySQL 8	`hospital/backend/src/db.js`	`HOSPITAL_DB_USER`, `HOSPITAL_DB_PASSWORD` y opcionales
Laboratorio Clínico	PostgreSQL	`laboratorio/backend/db.js`	`LABORATORIO_DB_URL`
Clínica Odontológica	Cloud Firestore	`odontologia/backend/db.js`	`ODONTOLOGIA_FIREBASE_CREDENCIALES` (ruta a la llave, fuera del repositorio)

5. Cómo está guardada la información

La base `centro_pediatrico_db` tiene cinco colecciones. Cada paciente es **un solo documento** que contiene su historial completo, de modo que abrir un expediente es una sola lectura.



Colección	Guarda	Por qué así
`pacientes`	Datos, tutores, consultas, controles y vacunas	Siempre se consulta el historial completo; cada cambio es atómico
`medicos`	Catálogo con colegiado y especialidad	Un médico atiende a muchos pacientes: se evita duplicarlo
`vacunas`	Catálogo de vacunas	Compartido por todos los esquemas
`usuarios`	Cuentas: usuario, rol y contraseña cifrada	Separada de los datos clínicos; nunca se devuelve a la interfaz
`bitacora_acceso`	Quién leyó o cambió qué y cuándo	Crece sin límite y se consulta por separado

Índices

Están definidos en la lista `INDICES` de `adapters/mongo/index.js`.

Colección	Campos	Tipo	Para qué
`pacientes`	`id_paciente`	Único	Identificar sin repetidos
`pacientes`	`tutores.dpi`	Normal	Buscar por DPI
`pacientes`	`estado_historial, apellidos, nombres`	Normal	Listados
`pacientes`	`bitacora_pendiente.id_evento`	Disperso	Localizar auditoría pendiente
`medicos`	`id_medico` · `colegiado`	Únicos	Referencias y colegiado
`vacunas`	`id_vacuna`	Único	Referencias

Colección	Campos	Tipo	Para qué
`usuarios`	`usuario`	Único	Inicio de sesión
`bitacora_acceso`	`id_paciente, fecha_hora` · `fecha_hora`	Normales	Consultar auditoría
`bitacora_acceso`	`id_evento`	Único parcial	No duplicar eventos entregados

Validación en dos niveles

- **En la API:** `backend/schema.js` (Zod) rechaza datos incorrectos antes de tocar la base, incluidos campos desconocidos.
- **En MongoDB:** `adapters/mongo/validadores.js` define reglas `$jsonSchema` que protegen la base aunque alguien escriba sin pasar por la API. Se instalan con `npm run db:validadores -- --aplicar`.

Cómo no se pierde la auditoría

Cada cambio clínico debe quedar registrado en la bitácora. Pero el cambio va en `pacientes` y el registro en `bitacora_acceso`, y un MongoDB standalone no puede garantizar una escritura en dos colecciones a la vez. `adapters/mongo/unidadDeTrabajo.js` lo resuelve así:

1. El evento viaja dentro del **mismo** `updateOne` que el cambio, en `bitacora_pendiente`. Se guardan los dos o ninguno.
2. Después se copia a `bitacora_acceso`. El índice único de `id_evento` impide duplicarlo.
3. Se retira del paciente con `$pull`.
4. Si la copia falla, el evento sigue en el paciente y `reenviarPendientes` lo entrega al iniciar, cada 60 segundos y antes de consultar la auditoría.

Si `MONGO_URI` apunta a un replica set, el mismo código usa una transacción real. Lo detecta solo con el comando `hello`. Las lecturas siguen otra regla: si no se puede registrar su evento, la API responde `503` y no entrega los datos.

6. Cómo viaja una petición

Ejemplo: un médico guarda una consulta nueva

#	Archivo	Qué ocurre
1	`paginas/NuevaConsulta.jsx`	Reúne el formulario y llama a `api.consultas.crear(id, cuerpo)`
2	`pediatria/frontend/api.js`	Traduce la llamada a `POST /api/pacientes/P001/consultas`
3	`plataforma/api/http.js`	Agrega `Authorization: Bearer <token>` y envía JSON
4	`apps/web/vite.config.js`	Solo en desarrollo: reenvía del puerto 5173 al 3001
5	`backend/app.js`	Cabeceras de seguridad, máximo 100 KB y límite de solicitudes
6	`api/enrutadorInstituciones.js`	Verifica el JWT y elige el módulo según `institucion`
7	`security.js` (`requireAuth`)	Comprueba rol e institución; deja el usuario en `req.actor`
8	`api/routes/pacientes.js`	Exige `clinico.escribir` y valida con `S.consulta` (Zod)

#	Archivo	Qué ocurre
9	`application/consultasService.js`	Verifica médico, historial activo y fechas; arma la consulta
10	`application/comun.js` (`escribir`)	Crea el evento de auditoría y abre la unidad de trabajo
11	`adapters/mongo/repositorios.js`	`updateOne` con condiciones y `$push` de consulta y evento
12	`adapters/mongo/unidadDeTrabajo.js`	Copia el evento a `bitacora_acceso` y lo retira del paciente
13	Respuesta	`201 { id_consulta, id_control }` regresa hasta la página

Cómo se inicia sesión

#	Archivo	Qué ocurre
1	`paginas/Login.jsx`	Envía usuario y contraseña a `POST /auth/login`
2	`application/autenticacionService.js`	Busca la cuenta en `usuarios`, compara la contraseña cifrada y registra el intento; `app.js` devuelve un JWT de 8 horas
3	`sesion/SesionContext.jsx`	Con el token, llama a `GET /api/sesion`
4	`api/routes/generales.js`	Responde usuario, rol, institución, permisos, expiración y capacidades
5	`SesionContext.jsx`	Guarda el JWT en `sessionStorage` y busca el módulo con `resolverSesion`
6	`portal/Portal.jsx`	Muestra las instituciones; «Ingresar» navega al módulo
7	`src/App.jsx`	Monta el layout y las rutas del módulo que informó el backend

Cuentas y contraseñas
Las cuentas iniciales se crean con `npm run db:crear-accesos`. En la base solo queda la contraseña cifrada con scrypt. El responsable del proyecto comparte los datos de acceso por un medio privado: **no están en este manual ni en Git**.

Un solo formulario para las cuatro instituciones

`POST /auth/login`, en `pediatria/backend/app.js`, decide dónde validar la cuenta **por el prefijo del usuario**. Cada institución guarda sus propias cuentas y devuelve un token de esa institución; por eso cada usuario entra solo a la suya y el portal marca las demás como «Sin autorización».

Usuario	Se valida en	Cómo se crean las cuentas
Sin prefijo (`admin`, `medico`…)	MongoDB, colección `usuarios`	`npm run db:crear-accesos`
`hosp.*`	MySQL, tabla `usuario`	`npm run hospital:usuario -- hosp.<nombre> <contraseña> <rol>`
`lab.*`	PostgreSQL, tabla `cuenta_acceso`	`npm run lab:usuarios`
`odonto.*`	Firestore	`npm run odonto:usuarios`

Qué pasa cuando algo falla

El servicio o el repositorio lanza un error (`domain/errors.js`). `manejadorErrores`, en `api/middleware.js`, lo convierte en `{ error, codigo, detalles? }`. `http.js` lo recibe como `ApiError`, cierra la sesión si es `401`, y la página lo muestra con `ErrorState` o `Alerta`. Los errores `500` nunca revelan detalles internos.

Estado	Código	Significa
400	`VALIDACION` · `REGLA_NEGOCIO`	Datos inválidos o regla incumplida
401	`CREDENCIALES_INVALIDAS`	Usuario o contraseña incorrectos
401	`NO_AUTENTICADO` · `TOKEN_INVALIDO` · `TOKEN_EXPIRADO`	Sin sesión válida
403	`ROL_SIN_PERMISO` · `INSTITUCION_NO_REGISTRADA` · `PROHIBIDO`	No tiene acceso
404	`NO_ENCONTRADO` · `RUTA_NO_ENCONTRADA`	No existe
409	`CONFLICTO`	Duplicado, archivado o cambio simultáneo
422	`REFERENCIA_INVALIDA` · `VALIDACION_MOTOR`	Médico o vacuna inexistente, o rechazo del validador
503	`AUDITORIA_NO_DISPONIBLE`	No se pudo registrar el acceso

7. Cómo está organizado el código

Backend: arquitectura hexagonal

La idea es separar las reglas del sistema de la tecnología. Si Pediatría cambiara de motor, solo se reescribiría el adaptador. Las pruebas aprovechan esto: cambian MongoDB por `tests/doubles/memoria.js` sin tocar los casos de uso.

Capa	Carpeta	Responsabilidad	Conoce a MongoDB
HTTP	`app.js`, `api/`, `schema.js`, `security.js`	Recibir, autorizar, validar, responder	No
Casos de uso	`application/`	Orquestar cada operación	No
Dominio	`domain/`	Reglas de negocio y errores	No
Puertos	`ports/index.js`	Lista de operaciones de datos necesarias	No
Adaptador	`adapters/mongo/`, `db.js`	Implementar los puertos con MongoDB	**Sí**
Composición	`server.js`	Unir todo al arrancar	Solo la conecta

`crearServicios` llama a `verificarRepositorios`: si un adaptador no implementa todos los métodos del puerto, el sistema falla al arrancar con un mensaje claro.

Frontend: una plataforma y módulos enchufables

1. `plataforma/modulos/registro.js` lista los módulos conectados.
2. Cada módulo describe sus rutas, menú y permisos en su `modulo.js`, sin escribir componentes. `contrato.js` lo valida.
3. Tras el inicio de sesión, `resolverSesion` toma el módulo de la institución **que informó el backend**.
4. `modulo.crearApi(http)` crea las operaciones, y `RutasModulo.jsx` carga cada página cuando se necesita.
5. Las páginas obtienen todo con `useModulo()`, y cargan o envían datos con `useCarga` y `useEnvio`.

El expediente pediátrico (`paginas/Expediente.jsx`) carga una vez al paciente y reparte sus pestañas: Datos generales y tutores, Consultas, Prescripciones, Crecimiento, Vacunación y Auditoría (carpeta `paginas/expediente/`).

Reglas entre carpetas

- Un módulo importa su carpeta con rutas relativas y lo común con `@plataforma/...`. **Nunca** archivos de otra institución.
- Ocultar un botón no es seguridad: el backend vuelve a autorizar cada petición.
- La vista previa (`/vista-previa`) solo existe con `vite`, no usa sesión y no puede guardar.

8. Mapa de rutas: de la pantalla a la colección

Cada fila sigue una operación completa. Las rutas `/:id` van bajo `/api/pacientes`. Toda ruta que toca un historial también registra su evento en `bitacora_acceso`.

Ruta	Permiso	Pantalla (página)	Servicio → repositorio	Colección
`GET /salud`	Público	Verificación manual	`app.js` directo	Ninguna
`POST /auth/login`	Público	`Login`	`autenticacion.iniciarSesion` → `obtenerPorUsuario`	usuarios, bitácora
`GET /api/sesion`	Cualquier rol	`SesionContext`	`capacidades`	Metadatos
`GET /api/panel`	`panel.ver`	`Panel`, `Pacientes`	`panel.resumen` → `resumen`, `dosisPendientes`	pacientes
`GET /api/pacientes`	`pacientes.listar`	`Pacientes`, `Archivados`	`listar` → `listar`	pacientes
`POST /api/pacientes`	`pacientes.registrar`	`PacienteNuevo`	`registrar` → `buscarDuplicado`, `siguienteId`, `crear`	pacientes
`GET /:id`	`historial.leer`	`Expediente`, `NuevaConsulta`	`obtener` → `obtener`	pacientes
`GET /:id/administrativo`	`pacientes.ver_administrativo`	`Expediente` (secretaría)	`obtenerAdministrativo`	pacientes
`PATCH /:id`	`pacientes.editar`	`Resumen`	`actualizarDatos`	pacientes
`PATCH /:id/tutores`	`pacientes.editar`	Sin pantalla	`reemplazarTutores`	pacientes
`POST /:id/tutores`	`pacientes.editar`	`Resumen`	`agregarTutor` → `reemplazarTutores`	pacientes
`PUT /:id/tutores/:t`	`pacientes.editar`	`Resumen`	`actualizarTutor` → `reemplazarTutores`	pacientes
`PATCH /:id/archivar`	`historial.archivar`	`Expediente`	`archivar`	pacientes
`GET /:id/consultas`	`historial.leer`	`Consultas`	`listar` → `obtener`, `medicos.listar`	pacientes, medicos
`POST /:id/consultas`	`clinico.escribir`	`NuevaConsulta`	`registrar` → `agregarConsulta`	pacientes
`POST …/diagnosticos`	`clinico.escribir`	`Consultas`	`agregarDiagnostico`	pacientes
`PUT …/tratamiento`	`clinico.escribir`	`Consultas`	`establecerTratamiento`	pacientes
`POST …/prescripciones`	`clinico.escribir`	`Consultas`, `Prescripciones`	`agregarPrescripcion`	pacientes
`GET /:id/prescripciones`	`historial.leer`	`Prescripciones`	`listarPrescripciones` → `obtener`	pacientes, medicos
`GET /:id/diagnosticos`	`historial.leer`	Sin pantalla	`listarDiagnosticos` → `obtener`	pacientes
`GET /:id/crecimiento`	`historial.leer`	`Crecimiento`	`historial` → `obtener`	pacientes

Ruta	Permiso	Pantalla (página)	Servicio → repositorio	Colección
`POST /:id/crecimiento`	`clinico.escribir`	`Crecimiento`	`registrar` → `agregarControl`	pacientes
`GET /:id/vacunacion`	`historial.leer`	`VacunacionPaciente`	`esquema` → `obtener`	pacientes, vacunas, medicos
`POST /:id/vacunacion`	`clinico.escribir`	`VacunacionPaciente`	`programar` → `programarDosis`	pacientes
`PATCH …/:e/aplicar`	`clinico.escribir`	`VacunacionPaciente`	`aplicar` → `aplicarDosis`	pacientes
`PATCH …/:e/cancelar`	`clinico.escribir`	`VacunacionPaciente`	`cancelar` → `cancelarDosis`	pacientes
`GET /api/vacunacion/pendientes`	`historial.leer`	`Vacunacion`	`pendientes` → `dosisPendientes`	pacientes, vacunas
`GET /api/medicos`	`catalogos.ver`	`Medicos`, `Expediente`, `NuevaConsulta`	`listarMedicos`	medicos
`GET /api/medicos/:id`	`catalogos.ver`	Sin pantalla	`obtenerMedico`	medicos
`POST /api/medicos`	`catalogos.editar`	Sin pantalla	`crearMedico`	medicos
`GET /api/especialidades`	`catalogos.ver`	`Medicos`	`especialidades`	medicos
`GET /api/vacunas`	`catalogos.ver`	`Vacunacion`, `Expediente`	`listarVacunas`	vacunas
`POST /api/vacunas`	`catalogos.editar`	Sin pantalla	`crearVacuna`	vacunas
`GET /api/auditoria`	`auditoria.leer`	`Auditoria`, `AuditoriaPaciente`	`auditoria.listar`	bitácora
`GET /api/auditoria/:id`	`auditoria.leer`	Sin pantalla	`auditoria.listar`	bitácora

«Sin pantalla» quiere decir que la ruta funciona, pero ninguna página la llama: médicos y vacunas se dan de alta directamente por la API, como indica el README de Pediatría.

9. Roles y permisos

La tabla está en `backend/permisos.js`. El backend la usa para autorizar cada ruta y la envía al navegador en `/api/sesion`, que solo la usa para decidir qué mostrar.

Permiso	Qué permite	Médico	Secretaría	Admin.
`panel.ver`	Panel principal	Sí	Sí	Sí
`pacientes.listar`	Buscar pacientes	Sí	Sí	Sí
`pacientes.ver_administrativo`	Datos generales y tutores	Sí	Sí	Sí
`pacientes.registrar`	Dar de alta	—	Sí	Sí
`pacientes.editar`	Editar datos y tutores	—	Sí	Sí
`historial.leer`	Información clínica	Sí	—	Sí
`historial.leer_archivados`	Historiales archivados	—	—	Sí

Permiso	Qué permite	Médico	Secretaría	Admin.
`historial.archivar`	Archivar	—	—	Sí
`clinico.escribir`	Consultas, recetas, crecimiento, vacunas	Sí	—	—
`catalogos.ver`	Ver médicos y vacunas	Sí	Sí	Sí
`catalogos.editar`	Crear médicos y vacunas	—	—	Sí
`auditoria.leer`	Bitácora	—	—	Sí

10. Dónde está cada control de seguridad

Control	Archivo
Verificación del JWT (HS256, emisor, audiencia, expiración)	`backend/security.js`
Permiso por ruta	`api/routes/*` con `permisos.js`
Datos de entrada estrictos	`backend/schema.js`
Máximo 100 KB por petición y cabeceras seguras	`backend/app.js`, `api/middleware.js`
Contraseñas cifradas y misma respuesta si la cuenta no existe	`contrasenas.js`, `autenticacionService.js`
Límite de solicitudes (600 por minuto en `/api`, 20 en inicio de sesión)	`middleware.js` (`limitador`)
DPI enmascarado para el médico	`comun.js` (`protegerTutores`), `reglas.js`
Secretaría sin datos clínicos	`pacientesService.js`, `permisos.js`
Historial archivado de solo lectura	Filtro `activo(id)` en `repositorios.js`
Auditoría de lecturas y escrituras	`comun.js`, `unidadDeTrabajo.js`
Credenciales fuera del código	`.env`, excluido en `.gitignore`

11. Catálogo de archivos

Los 295 archivos versionados del repositorio, agrupados por carpeta, más este manual. En cada tabla, la ruta es relativa a la carpeta indicada. No se incluyen carpetas generadas, como `node_modules/` o `dist/`.

Raíz del repositorio

Archivo	Qué hace	Se comunica con
`.env.example`	Plantilla de las variables de entorno, sin secretos reales.	Se copia a `.env`, que leen `server.js` y los scripts.
`.gitattributes`	Fija el fin de línea LF y marca PDF e imágenes como binarios.	Git.
`.gitignore`	Evita subir `.env`, `node_modules`, `dist`, respaldos y volcados de bases.	Git.
`README.md`	Presentación, instalación, comandos y problemas comunes.	Enlaza a `docs/` y a los README de cada institución.
`package.json`	Dependencias del backend y las pruebas; define **todos** los comandos `npm`.	Ejecuta `server.js`, los scripts y las pruebas; delega en `apps/web`.
`package-lock.json`	Versiones exactas de las dependencias de la raíz.	`npm ci`.

docs/ — documentación común

`docs/`

Archivo	Qué hace	Se comunica con
`DISENO.md`	Sistema visual: colores, temas, componentes y accesibilidad.	Describe `styles.css`, `mockup.css` y `plataforma/ui/`.
`GUIA_EQUIPO.md`	Contratos entre instituciones (sesión, HTTP, errores, permisos) y cómo conectar una.	Referencia para las cuatro instituciones.
`MANUAL_DEL_PROYECTO.pdf`	Este manual.	—

apps/web/ — configuración de la aplicación web

`apps/web/`

Archivo	Qué hace	Se comunica con
`.gitignore`	Exclusiones propias de la aplicación web.	Git.
`alias.js`	Define los alias `@plataforma` e `@instituciones` en un solo lugar.	`vite.config.js` y `tests/soporte/alias-frontend-ganchos.mjs`.
`index.html`	Página base. Aplica el tema antes de cargar React para evitar parpadeos.	Carga `src/main.jsx`.
`package.json`	Dependencias del frontend (React, React Router, Vite).	`npm --prefix apps/web`.
`package-lock.json`	Versiones exactas del frontend.	`npm run web:ci`.

Archivo	Qué hace	Se comunica con
`vite.config.js`	Servidor de desarrollo (5173) y compilación. **Redirige `/api`, `/auth` y `/salud` a Express.**	Reenvía a `localhost:3001` (o `VITE_API_PROXY`). Usa `alias.js`.

apps/web/src/ — arranque de React y estilos

`apps/web/src/`

Archivo	Qué hace	Se comunica con
`main.jsx`	Punto de entrada de React: monta enrutador, sesión, notificaciones y estilos.	`App.jsx`, `SesionContext.jsx`, `registro.js`, `tema.js`.
`App.jsx`	Rutas principales: `/login`, `/instituciones`, `/vista-previa/*` y la zona protegida.	`useSesion`, `Portal`, `Login`, `AppLayout`, `RutasModulo`.
`styles.css`	Paleta, tokens por tema, componentes y comportamiento responsive.	Todas las pantallas.
`mockup.css`	Composición visual del mockup: menú navy, acceso dividido, encabezados de paciente.	Todas las pantallas; se carga tras `styles.css`.

plataforma/ — utilidades y cliente HTTP

`apps/web/src/plataforma/`

Archivo	Qué hace	Se comunica con
`api/http.js`	**Cliente HTTP único.** Agrega el token, envía JSON y convierte errores en `ApiError`.	Hace `fetch` al backend. Lo usan `SesionContext` y cada `api.js`.
`formato.js`	Formato de fechas, edad, nombre completo y conversión a ISO.	Páginas de los módulos.
`tema.js`	Tema claro, oscuro o según el sistema; se guarda en `localStorage`.	`main.jsx`, `UserMenu`, `ControlesVistaPrevia`.

plataforma/layout/ — estructura visual común

`apps/web/src/plataforma/layout/`

Archivo	Qué hace	Se comunica con
`AppLayout.jsx`	Armazón de la pantalla: menú lateral, encabezado y contenido.	`Sidebar`, `Header`.
`Sidebar.jsx`	Menú lateral con las secciones que los permisos permiten.	`useModulo`, `navegacion` del manifiesto.
`Header.jsx`	Encabezado: migas de pan, acción rápida, institución y usuario.	`Breadcrumbs`, `InstitutionIndicator`, `UserMenu`.
`Breadcrumbs.jsx`	Migas de pan a partir de los títulos del manifiesto.	`migasPara` en `contrato.js`.
`InstitutionIndicator.jsx`	Muestra la institución que el servidor asignó a la sesión.	`useSesion`.
`UserMenu.jsx`	Menú de usuario: rol, expiración, tema y cierre de sesión.	`useSesion`, `tema.js`.

plataforma/modulos/ — cómo se conectan las instituciones

`apps/web/src/plataforma/modulos/`

Archivo	Qué hace	Se comunica con
`contrato.js`	Reglas que debe cumplir un módulo; resuelve qué módulo corresponde a una sesión.	`registro.js`, `SesionContext`, `catalogo.js`, `portal.js`.
`registro.js`	**Lista de instituciones conectadas.** Hoy solo Pediatría.	Importa `pediatria/frontend/modulo.js`.
`ModuloContext.jsx`	Da a las páginas `api`, `puede` y `enlace` con `useModulo()`.	`App.jsx`, `VistaPrevia.jsx` y todas las páginas.
`RutasModulo.jsx`	Crea las rutas del manifiesto y carga cada página bajo demanda.	`useModulo`, `SinPermiso`.

plataforma/paginas/, portal/ y sesion/

`apps/web/src/plataforma/`

Archivo	Qué hace	Se comunica con
`paginas/Login.jsx`	Pantalla de acceso con usuario y contraseña.	`useSesion`; `POST /auth/login`.
`paginas/EstadosSesion.jsx`	Mensajes «Sin permiso» e «Institución no disponible».	`RutasModulo`, `App.jsx`.
`portal/Portal.jsx`	Pantalla para elegir institución tras iniciar sesión. Solo navega.	`portal.js`, `presentaciones.js`, `registro.js`.
`portal/portal.js`	Calcula el estado de cada tarjeta y el destino tras el inicio de sesión.	`contrato.js`.
`portal/presentaciones.js`	Descubre el `presentacion.js` de cada institución.	`instituciones/*/frontend/presentacion.js`.
`sesion/SesionContext.jsx`	**Centro de la sesión.** Guarda el JWT, llama a `/api/sesion` y crea la API del módulo.	`http.js`, `contrato.js`; lo usa toda la interfaz.

plataforma/ui/ — componentes reutilizables

`apps/web/src/plataforma/ui/`

Archivo	Qué hace	Se comunica con
`index.js`	Punto único desde donde se importan los componentes.	Todas las páginas (`@plataforma/ui/index.js`).
`hooks.js`	`useCarga` (traer datos) y `useEnvio` (enviar sin doble clic).	Páginas que llaman a la API.
`Datos.jsx`	Tablas, indicadores, estados, pares dato-valor y paginación.	Páginas de listado y paneles.
`Dialogo.jsx`	Ventana modal nativa y accesible.	Formularios y confirmaciones.
`Estados.jsx`	Pantallas de carga, vacío, error, alerta y «pendiente».	Todas las páginas.

Archivo	Qué hace	Se comunica con
`FlujoClinico.jsx`	Secciones numeradas, pasos y resumen de formularios.	Formularios clínicos.
`Formularios.jsx`	Campo accesible, botón de envío, búsqueda y filtros.	Formularios y listados.
`Icon.jsx`	Iconos propios, sin dependencias externas.	Toda la interfaz.
`Notificaciones.jsx`	Avisos emergentes y confirmaciones (`useNotificar`, `useConfirmar`).	Montado en `main.jsx`.
`Pagina.jsx`	Encabezado de página, tarjeta y pestañas.	Todas las páginas.
`VistasInstitucionales.jsx`	Bloques compartidos de los diseños de vista previa.	Módulos pendientes.

plataforma/vista-previa/ — solo en desarrollo

`apps/web/src/plataforma/vista-previa/`

Archivo	Qué hace	Se comunica con
`VistaPrevia.jsx`	Muestra módulos pendientes sin sesión ni backend.	`catalogo.js`, `RutasModulo`, `AppLayout`.
`catalogo.js`	Descubre los módulos y sabe cuáles están conectados.	`instituciones/*/frontend/modulo.js`, `registro.js`.
`EstadoIntegracion.jsx`	Portada con el avance de cada institución.	`catalogo.js`.
`AccionVistaPrevia.jsx`	Botón «Ver diseño» en el portal.	`Portal.jsx`, `catalogo.js`.
`ControlesVistaPrevia.jsx`	Selector de tema y regreso, en la vista previa.	`tema.js`.

Pediatría · backend: arranque y conexión

`instituciones/pediatria/backend/`

Archivo	Qué hace	Se comunica con
`db.js`	**Abre la conexión con MongoDB.** Crea el `MongoClient` y elige la base.	Lee `MONGO_URI` y `MONGO_DB`. Lo usan `server.js` y `seed.js`.
`server.js`	**Punto de entrada.** Conecta, crea índices, arma los servicios y escucha.	`db.js`, `adapters/mongo`, `application`, `app.js`.
`app.js`	Construye Express: seguridad, `/salud`, `/auth/login`, `/api` y sirve la web compilada.	`api/*`, `security.js`, `apps/web/dist`.
`schema.js`	Valida con Zod los datos que entran a la API.	`api/routes/*`, `app.js`.
`security.js`	Verifica el JWT, exige rol e institución y firma la sesión.	`app.js`, `enrutadorInstituciones.js`.
`contrasenas.js`	Cifra y verifica contraseñas con scrypt.	`autenticacionService.js`, `crear-accesos.js`.
`permisos.js`	**Tabla rol → permisos**, única fuente de verdad.	`api/routes/*`; se envía al navegador en `/api/sesion`.

Pediatría · backend: capa HTTP

`instituciones/pediatria/backend/api/`

Archivo	Qué hace	Se comunica con
`enrutadorInstituciones.js`	Lee la institución del token y envía la petición a su módulo.	`security.js`; montado en `app.js`.
`middleware.js`	Cabeceras de seguridad, CORS, límite de solicitudes y manejo de errores.	`app.js`, `api/routes/*`.
`routes/pacientes.js`	Rutas `/api/pacientes/...`: valida y llama al caso de uso.	`schema.js`, `permisos.js`, servicios de `application/`.
`routes/generales.js`	Rutas de sesión, panel, catálogos, pendientes y auditoría.	`schema.js`, `permisos.js`, servicios.

Pediatría · backend: dominio, casos de uso y puertos

`instituciones/pediatria/backend/`

Archivo	Qué hace	Se comunica con
`domain/errors.js`	Errores de negocio con su código HTTP.	Servicios, repositorios y `manejadorErrores`.
`domain/reglas.js`	Reglas de negocio puras: fechas, tutores, ids, DPI enmascarado.	Servicios y repositorios.
`ports/index.js`	**Contrato** de los métodos que debe tener cualquier adaptador de datos.	`application/index.js`, adaptador Mongo y doble en memoria.
`application/index.js`	Une los casos de uso; incluye catálogos, panel y auditoría.	Recibe repositorios; lo usa `server.js`.
`application/comun.js`	Auditoría, lectura de historial, escritura con evento, protección del DPI.	Puertos `pacientes`, `auditoria`, `unidad`.
`application/autenticacionService.js`	Inicio de sesión: valida la contraseña y registra el intento.	Puerto `usuarios`, `contrasenas.js`, `comun.js`.
`application/pacientesService.js`	Registrar, listar, editar, tutores y archivar.	Puerto `pacientes`, `comun.js`, `reglas.js`.
`application/consultasService.js`	Consultas, diagnósticos, tratamientos y recetas.	Puertos `pacientes` y `medicos`.
`application/crecimientoService.js`	Controles de peso, talla y perímetro cefálico.	Puerto `pacientes`.
`application/vacunacionService.js`	Programar, aplicar y cancelar dosis; pendientes.	Puertos `pacientes`, `vacunas`, `medicos`.

Pediatría · backend: adaptador MongoDB

`instituciones/pediatria/backend/adapters/mongo/`

Archivo	Qué hace	Se comunica con
`index.js`	Define e instala los índices, detecta capacidades y arma los repositorios.	`repositorios.js`, `unidadDeTrabajo.js`, `server.js`, `seed.js`.
`repositorios.js`	**Aquí están las consultas reales a MongoDB** (`find`, `updateOne`, `aggregate`).	Colecciones `pacientes`, `medicos`, `vacunas`, `usuarios`, `bitacora_acceso`.
`unidadDeTrabajo.js`	Guarda cada cambio junto con su auditoría: transacción u outbox.	`pacientes`, `bitacora_acceso`; lo usa `repositorios.js`.
`validadores.js`	Reglas `$jsonSchema` que la propia base aplica a sus colecciones.	`administracionValidadores.js`.
`administracionValidadores.js`	Verifica, respalda, aplica y revierte validadores.	`scripts/validadores.js`.

Pediatría · frontend: configuración del módulo

`instituciones/pediatria/frontend/`

Archivo	Qué hace	Se comunica con
`modulo.js`	**Manifiesto:** rutas, menú, roles y permisos del módulo.	`registro.js`; carga `api.js` y `paginas/`.
`api.js`	**Todas las rutas del backend que usa la interfaz pediátrica.**	`http.js` → API Express.
`presentacion.js`	Nombre, sigla, descripción e icono del portal.	`modulo.js`, `presentaciones.js`.

Pediatría · frontend: páginas

`instituciones/pediatria/frontend/paginas/`

Archivo	Qué hace	Se comunica con
`Panel.jsx`	Inicio con indicadores y consultas recientes.	`api.panel`.
`Pacientes.jsx`	Búsqueda y listado; incluye `ListadoPacientes`.	`api.pacientes.listar`, `api.panel`.
`PacienteNuevo.jsx`	Formulario de registro con tutores.	`api.pacientes.crear`.
`Expediente.jsx`	Expediente con pestañas; también archiva.	`api.pacientes.obtener` / `administrativo` / `archivar`, catálogos.
`NuevaConsulta.jsx`	Consulta con diagnósticos, receta y control.	`api.consultas.crear`.
`Vacunacion.jsx`	Catálogo de vacunas y dosis pendientes.	`api.catalogos.vacunas`, `api.vacunacion.pendientes`.
`Medicos.jsx`	Catálogo de médicos por especialidad.	`api.catalogos.medicos`, `especialidades`.
`Archivados.jsx`	Historiales archivados (solo administrador).	`ListadoPacientes` de `Pacientes.jsx`.

Archivo	Qué hace	Se comunica con
`Auditoria.jsx`	Bitácora con filtros.	`api.auditoria`.
`expediente/Resumen.jsx`	Pestaña de datos generales y tutores.	`api.pacientes.actualizar`, `agregarTutor`, `actualizarTutor`.
`expediente/Consultas.jsx`	Pestaña de consultas, diagnósticos, tratamientos y recetas.	`api.consultas.*`.
`expediente/Prescripciones.jsx`	Pestaña de medicamentos históricos.	`api.consultas.prescripciones`.
`expediente/Crecimiento.jsx`	Pestaña de mediciones y gráficas.	`api.crecimiento.*`, `GraficaCrecimiento`.
`expediente/VacunacionPaciente.jsx`	Pestaña del esquema de vacunación.	`api.vacunacion.*`.
`expediente/AuditoriaPaciente.jsx`	Pestaña de auditoría del paciente.	`api.auditoria`.

Pediatría · frontend: componentes

`instituciones/pediatria/frontend/componentes/`

Archivo	Qué hace	Se comunica con
`ClinicoCampos.jsx`	Editores de diagnósticos y medicamentos; selector de médico.	`NuevaConsulta`, `Consultas`.
`GraficaCrecimiento.jsx`	Gráficas de lo medido; sin percentiles ni curvas de referencia.	`Crecimiento.jsx`.
`TutorCampos.jsx`	Campos y validación del tutor; lo convierte al formato de la API.	`PacienteNuevo`, `Resumen`.

Pediatría · scripts de operación

`instituciones/pediatria/scripts/`

Archivo	Qué hace	Se comunica con
`seed.js`	Crea índices faltantes. **No inserta datos**, pese a su nombre (`npm run db:indices`).	`db.js`, `adapters/mongo`.
`validadores.js`	Verifica, aplica o revierte validadores (`npm run db:validadores`).	MongoDB (`MONGO_URI` o `MONGO_ADMIN_URI`).
`crear-accesos.js`	Crea las cuentas iniciales y `CREDENCIALES_ACCESO.txt` (`npm run db:crear-accesos`).	`db.js`, `repositorios.js`, `contrasenas.js`.
`respaldo/comun.js`	Huellas SHA-256, integridad y localización de herramientas.	Scripts de respaldo.
`respaldo/diagnostico.js`	Informe de solo lectura de la base.	MongoDB (`MONGO_URI`).
`respaldo/respaldar.js`	Respaldo BSON con `mongodump` y manifiesto.	MongoDB y `mongodump`; escribe en `respaldos/`.

Archivo	Qué hace	Se comunica con
`respaldo/restaurar-aislado.js`	Restaura en un `mongod` temporal y compara.	`mongodTemporal.js`, `mongorestore`.

Pediatría · documentación

`instituciones/pediatria/`

Archivo	Qué hace	Se comunica con
`README.md`	API, roles, seguridad, pruebas y limitaciones.	Punto de partida de Pediatría.
`docs/MODELO_Y_DECISIONES.md`	Modelo de datos, índices y justificación de decisiones.	Documento comparativo.
`docs/OPERACION_MONGODB.md`	Validadores, outbox y plan de replica set.	`scripts/validadores.js`.
`docs/RESPALDO_Y_RESTAURACION.md`	Procedimiento de respaldo y restauración verificada.	`scripts/respaldo/*`.
`docs/TRAZABILIDAD_REQUISITOS.md`	Matriz entre requisitos del enunciado y lo implementado.	Enunciado y Fase I.
`docs/DEMOSTRACION.md`	Guion para presentar el proyecto.	Interfaz y MongoDB Compass.
`docs/AUDITORIA_2026-10-31.md`	Auditoría técnica: operaciones, defectos y preguntas de defensa.	Código de Pediatría.
`docs/Informe_Fase2_Centro_Pediatrico.pdf`	Informe técnico de la Entrega II.	Entrega académica.
`docs/Informe_Fase2_Centro_Pediatrico.docx`	Versión editable del informe.	Entrega académica.

Pediatría · pruebas

`instituciones/pediatria/tests/`

Archivo	Qué hace	Se comunica con
`helpers.js`	Arranca la API real sobre el doble en memoria.	`application`, `app.js`, `doubles/memoria.js`.
`doubles/memoria.js`	Imita los puertos en memoria, sin MongoDB.	Cumple `ports/index.js`.
`api.test.js`	Flujo HTTP completo, permisos, auditoría y errores.	`helpers.js`.
`dominio.test.js`	Reglas de dominio y contratos de puertos.	`domain/`, `ports/`.
`schema.test.js`	Esquemas de entrada Zod.	`schema.js`.
`permisos.test.js`	Lo que se anuncia al frontend coincide con lo que se autoriza.	`permisos.js`, API.
`seguridad.test.js`	JWT, DPI enmascarado, parámetros y rutas restringidas.	API.
`enrutamiento.test.js`	La institución del token elige el módulo.	`enrutadorInstituciones.js`.
`hallazgos-auditoria.test.js`	Regresión de hallazgos de la auditoría técnica.	API sobre el doble.

Archivo	Qué hace	Se comunica con
`identificador-paciente.test.js`	Identificador automático `P###` (defecto D-01).	Doble y MongoDB desechable.
`integridad-auditoria.test.js`	Duplicados, carreras y fallos de bitácora.	Servicios y doble.
`mongo.integration.test.js`	Adaptador contra **MongoDB real desechable**.	`mongodTemporal.js`.
`soporte/mongodTemporal.js`	Levanta un `mongod` propio en puerto libre y lo elimina.	`mongod`; pruebas y restauración.
`soporte/validadorInstalado.js`	Copia de un validador anterior para reproducirlo en pruebas.	`mongo.integration.test.js`.
`soporte/carrerasNacimiento.js`	Escenario de cambios simultáneos de fecha de nacimiento.	Doble y MongoDB.
`e2e/flujo-ui.e2e.js`	Recorre la interfaz en Edge o Chrome con MongoDB desechable.	`playwright-core`, app compilada, `auditoria-ui.js`.

Hospital General (MySQL)

`instituciones/hospital/`

Archivo	Qué hace	Se comunica con
`README.md`	Estado, puesta en marcha, acceso y roles del módulo.	`docs/GUIA_EQUIPO.md`.
`frontend/modulo.js`	Manifiesto: rutas, navegación y permisos. Registrado en la plataforma.	`registro.js`.
`frontend/api.js`	Llamadas HTTP a su backend.	`plataforma/api/http.js` y sus rutas.
`frontend/presentacion.js`	Datos de la tarjeta del portal.	`presentaciones.js`.
`frontend/componentes/EncabezadoExpediente.jsx`	Encabezado del expediente.	`Expediente.jsx`.
`frontend/paginas/Panel.jsx`	Indicadores y accesos con datos de su base.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Pacientes.jsx`	Búsqueda y listado de pacientes.	Su `api.js` y plataforma `ui`.
`frontend/paginas/PacienteNuevo.jsx`	Registro de un paciente.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Expediente.jsx`	Pestañas del expediente hospitalario.	`EncabezadoExpediente`, su `api.js`.
`frontend/paginas/NuevaConsulta.jsx`	Registro de una consulta.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Archivar.jsx`	Archiva el historial con su motivo.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Archivados.jsx`	Historiales archivados.	`Pacientes.jsx`.
`frontend/paginas/Citas.jsx`	Agenda de citas.	Su `api.js` y plataforma `ui`.

Archivo	Qué hace	Se comunica con
`frontend/paginas/SeccionHospital.jsx`	Elige la página según la ruta: hospitalización, catálogos o bitácora.	`Hospitalizacion`, `Catalogos`, `Bitacora`.
`backend/README.md`	Puesta en marcha: servidor común y modo independiente.	`package.json` de la raíz.
`backend/.env.example`	Plantilla del modo independiente, sin secretos.	`src/config.js`.
`docs/MODELO_Y_DECISIONES.md`	Modelo relacional, consistencia, índices y seguridad.	`backend/sql/01_esquema.sql`.
`tests/hospital.integration.test.js`	Login, permisos, pacientes, consultas, citas, hospitalización y bitácora.	`modulo.js`, MySQL desechable.
`tests/soporte/mysqlTemporal.js`	Levanta un MySQL desechable para las pruebas.	Pruebas de integración.
`backend/package.json`	Dependencias propias del backend (ESM).	`npm run hospital:install`.
`backend/package-lock.json`	Versiones exactas de esas dependencias.	`npm ci`.
`backend/scripts/crear-usuario.js`	Crea una cuenta con contraseña cifrada (bcrypt).	`npm run hospital:usuario`, tabla `usuario`.
`backend/sql/01_esquema.sql`	Crea la base `historial_clinico` y sus 29 tablas.	MySQL.
`backend/sql/02_datos_base.sql`	Roles obligatorios y catálogos ficticios.	MySQL.
`backend/src/modulo.js`	**Composición para el servidor común:** router, prefijo `hosp.` y `autenticar`.	`pediatria/backend/server.js`.
`backend/src/config.js`	Lee las variables `HOSPITAL_DB_*` y `JWT_*`.	`.env`.
`backend/src/db.js`	**Conexión a MySQL:** pool, consultas parametrizadas y transacciones.	Todas las rutas.
`backend/src/router.js`	Reúne las rutas bajo `/api` y responde `GET /sesion`.	`src/rutas/`, `seguridad.js`.
`backend/src/seguridad.js`	Verifica y firma el JWT; permisos por rol.	Todas las rutas.
`backend/src/bitacora.js`	Registra el acceso a tablas restringidas.	Tabla `bitacora_acceso`.
`backend/src/errores.js`	Formato de error común y traducción de errores de MySQL.	`modulo.js`, `app.js`.
`backend/src/utilidades.js`	Validación, paginación y campos reutilizables.	Rutas.
`backend/src/app.js`	Servidor independiente, solo para desarrollo.	`index.js`.
`backend/src/index.js`	Arranque del modo independiente (puerto 3002).	`app.js`.
`backend/src/rutas/auth.js`	Valida usuario y contraseña en MySQL.	`modulo.js`, tabla `usuario`.
`backend/src/rutas/panel.js`	Indicadores del panel.	MySQL.
`backend/src/rutas/pacientes.js`	Pacientes: listado, registro, edición y archivo.	`bitacora.js`, MySQL.
`backend/src/rutas/clinico.js`	Consultas, diagnósticos, recetas, exámenes y resultados.	`bitacora.js`, MySQL.

Archivo	Qué hace	Se comunica con
backend/src/rutas/citas.js	Agenda de citas y choques de horario.	MySQL.
backend/src/rutas/hospitalizacion.js	Camas, ingresos y altas.	MySQL.
backend/src/rutas/catalogos.js	Consulta y edición de catálogos.	MySQL.
backend/src/rutas/auditoria.js	Consulta de la bitácora.	Tabla bitacora_acceso.
frontend/componentes/BuscadorPaciente.jsx	Selector de paciente con búsqueda.	Páginas del módulo.
frontend/paginas/Bitacora.jsx	Consulta de la bitácora de acceso.	Su api.js y plataforma ui.
frontend/paginas/Catalogos.jsx	Administración de catálogos.	Su api.js y plataforma ui.
frontend/paginas/Hospitalizacion.jsx	Camas e ingresos hospitalarios.	Su api.js y plataforma ui.

Clínica Odontológica (Cloud Firestore)

instituciones/odontologia/

Archivo	Qué hace	Se comunica con
README.md	Estado, puesta en marcha, acceso y roles del módulo.	docs/GUIA_EQUIPO.md.
frontend/modulo.js	Manifiesto: rutas, navegación y permisos. Registrado en la plataforma.	registro.js.
frontend/api.js	Llamadas HTTP a su backend.	plataforma/api/http.js y sus rutas.
frontend/presentacion.js	Datos de la tarjeta del portal.	presentaciones.js.
frontend/componentes/EncabezadoExpediente.jsx	Encabezado del expediente.	Expediente.jsx.
frontend/componentes/PiezasDentales.jsx	Odontograma: estado de cada pieza dental.	Expediente.jsx.
frontend/paginas/Panel.jsx	Indicadores y accesos con datos de su base.	Su api.js y plataforma ui.
frontend/paginas/Pacientes.jsx	Búsqueda y listado de pacientes.	Su api.js y plataforma ui.
frontend/paginas/PacienteNuevo.jsx	Registro de un paciente.	Su api.js y plataforma ui.
frontend/paginas/Expediente.jsx	Pestañas del expediente dental.	PiezasDentales, su api.js.
frontend/paginas/NuevaConsulta.jsx	Registro de consulta y tratamiento por pieza.	Su api.js y plataforma ui.
frontend/paginas/Archivar.jsx	Archiva el historial con su motivo.	Su api.js y plataforma ui.

Archivo	Qué hace	Se comunica con
`frontend/paginas/Archivados.jsx`	Historiales archivados.	`Pacientes.jsx`.
`frontend/paginas/Citas.jsx`	Agenda de citas.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Administracion.jsx`	Catálogos y bitácora.	Su `api.js` y plataforma `ui`.
`backend/index.js`	**Composición:** conexión, adaptador, casos de uso, router y prefijo `odonto.` del login.	`pediatria/backend/server.js`.
`backend/app.js`	Router del módulo, montable en el servidor común.	`api/routes/`, `security.js`.
`backend/permisos.js`	Tabla única de permisos por rol.	`/api/sesion` y las rutas.
`backend/schema.js`	Validación de entrada con Zod.	`api/routes/`.
`backend/security.js`	Verifica y firma el JWT de la institución.	`app.js`, `index.js`.
`backend/contrasenas.js`	Cifrado y verificación con scrypt.	`accesoService.js`.
`backend/domain/errors.js`	Errores de negocio con su estado HTTP.	Casos de uso y rutas.
`backend/domain/reglas.js`	Reglas de negocio, sin depender del motor.	Casos de uso.
`backend/ports/index.js`	Contrato de persistencia.	Adaptador de Firestore, doble en memoria.
`backend/application/index.js`	Reúne los casos de uso.	`index.js`.
`backend/application/comun.js`	Bitácora, lectura de historiales y escrituras auditadas.	Casos de uso.
`backend/application/accesoService.js`	Inicio de sesión y alta de cuentas.	`contrasenas.js`, puerto de cuentas.
`backend/application/catalogosService.js`	Casos de uso de catálogos.	Puertos.
`backend/application/pacientesService.js`	Casos de uso de pacientes.	Puertos.
`backend/api/middleware.js`	Manejo de rutas asíncronas y formato de error común.	`api/routes/`.
`backend/api/routes/generales.js`	Sesión, panel, catálogos y bitácora.	Casos de uso.
`backend/api/routes/pacientes.js`	Rutas de pacientes y expediente.	Casos de uso.
`scripts/usuarios.js`	Crea una cuenta por rol y guarda las contraseñas fuera del repositorio.	`npm run odonto:usuarios`.
`docs/MODELO_Y_DECISIONES.md`	Modelo de datos y decisiones del motor.	—
`frontend/componentes/formato.js`	Utilidades de presentación, sin React.	Páginas del módulo.
`tests/api.test.js`	Rutas HTTP sobre el doble en memoria.	`app.js`.
`tests/dominio.test.js`	Reglas de negocio.	`domain/reglas.js`.
`tests/permisos.test.js`	Cada permiso coincide con la autorización real.	`permisos.js`.

Archivo	Qué hace	Se comunica con
`tests/servicios.test.js`	Casos de uso sobre el doble en memoria.	`application/`.
`tests/helpers.js`	Datos y tokens de prueba.	Pruebas.
`tests/doubles/memoria.js`	Doble en memoria de los puertos.	`ports/index.js`.
`tests/soporte/flujoComun.js`	Batería común para el doble y el motor real.	Pruebas.
`tests/soporte/flujoAcceso.js`	Batería común de cuentas de acceso.	Pruebas.
`backend/db.js`	**Conexión a Firestore** con la llave de la cuenta de servicio o el emulador.	`index.js`, `scripts/`.
`backend/adapters/firestore/index.js`	Adaptador: repositorios, `runTransaction` y traducción de errores.	`repositorios.js`.
`backend/adapters/firestore/repositorios.js`	Lecturas y escrituras de cada colección.	Cloud Firestore.
`backend/application/consultasService.js`	Casos de uso de consultas y tratamientos.	Puertos.
`backend/application/citasService.js`	Casos de uso de citas.	Puertos.
`tests/firestore.integration.test.js`	Integración con el emulador; se omite sin Java 21+.	`firestoreTemporal.js`.
`tests/soporte/firestoreTemporal.js`	Levanta el emulador de Firestore.	Pruebas de integración.

Laboratorio Clínico (PostgreSQL)

`instituciones/laboratorio/`

Archivo	Qué hace	Se comunica con
`README.md`	Estado, puesta en marcha, acceso y roles del módulo.	`docs/GUIA_EQUIPO.md`.
`frontend/modulo.js`	Manifiesto: rutas, navegación y permisos. Registrado en la plataforma.	`registro.js`.
`frontend/api.js`	Llamadas HTTP a su backend.	`plataforma/api/http.js` y sus rutas.
`frontend/presentacion.js`	Datos de la tarjeta del portal.	`presentaciones.js`.
`frontend/componentes/EncabezadoExpediente.jsx`	Encabezado del expediente.	`Expediente.jsx`.
`frontend/componentes/VisorResultado.jsx`	Informe del resultado, según el tipo de examen.	`DetalleResultado.jsx`.
`frontend/paginas/Panel.jsx`	Indicadores y accesos con datos de su base.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Pacientes.jsx`	Búsqueda y listado de pacientes.	Su `api.js` y plataforma `ui`.
`frontend/paginas/PacienteNuevo.jsx`	Registro de un paciente.	Su `api.js` y plataforma `ui`.

Archivo	Qué hace	Se comunica con
`frontend/paginas/Expediente.jsx`	Pestañas del expediente de laboratorio.	`EncabezadoExpediente`, su `api.js`.
`frontend/paginas/Solicitudes.jsx`	Listado de solicitudes de exámenes.	Su `api.js` y plataforma `ui`.
`frontend/paginas/NuevaSolicitud.jsx`	Crea una solicitud de exámenes.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Resultados.jsx`	Listado de resultados.	Su `api.js` y plataforma `ui`.
`frontend/paginas/RegistrarResultado.jsx`	Captura de los valores de un resultado.	Su `api.js` y plataforma `ui`.
`frontend/paginas/DetalleResultado.jsx`	Informe de un resultado y su validación.	`VisorResultado`, su `api.js`.
`frontend/paginas/Archivar.jsx`	Archiva el historial con su motivo.	Su `api.js` y plataforma `ui`.
`frontend/paginas/Archivados.jsx`	Historiales archivados.	`Pacientes.jsx`.
`frontend/paginas/Administracion.jsx`	Catálogos guiados y bitácora.	Su `api.js` y plataforma `ui`.
`backend/index.js`	**Composición:** conexión, adaptador, casos de uso, router y prefijo `lab.` del login.	`pediatria/backend/server.js`.
`backend/app.js`	Router del módulo, montable en el servidor común.	`api/routes/`, `security.js`.
`backend/permisos.js`	Tabla única de permisos por rol.	`/api/sesion` y las rutas.
`backend/schema.js`	Validación de entrada con Zod.	`api/routes/`.
`backend/security.js`	Verifica y firma el JWT de la institución.	`app.js`, `index.js`.
`backend/contrasenas.js`	Cifrado y verificación con scrypt.	`accesoService.js`.
`backend/domain/errors.js`	Errores de negocio con su estado HTTP.	Casos de uso y rutas.
`backend/domain/reglas.js`	Reglas de negocio, sin depender del motor.	Casos de uso.
`backend/ports/index.js`	Contrato de persistencia.	Adaptador de PostgreSQL, doble en memoria.
`backend/application/index.js`	Reúne los casos de uso.	`index.js`.
`backend/application/comun.js`	Bitácora, lectura de historiales y escrituras auditadas.	Casos de uso.
`backend/application/accesoService.js`	Inicio de sesión y alta de cuentas.	`contrasenas.js`, puerto de cuentas.
`backend/application/catalogosService.js`	Casos de uso de catálogos.	Puertos.
`backend/application/pacientesService.js`	Casos de uso de pacientes.	Puertos.
`backend/api/middleware.js`	Manejo de rutas asíncronas y formato de error común.	`api/routes/`.

Archivo	Qué hace	Se comunica con
`backend/api/routes/generales.js`	Sesión, panel, catálogos y bitácora.	Casos de uso.
`backend/api/routes/pacientes.js`	Rutas de pacientes y expediente.	Casos de uso.
`scripts/usuarios.js`	Crea una cuenta por rol y guarda las contraseñas fuera del repositorio.	`npm run lab:usuarios`.
`docs/MODELO_Y_DECISIONES.md`	Modelo de datos y decisiones del motor.	—
`frontend/componentes/formato.js`	Utilidades de presentación, sin React.	Páginas del módulo.
`tests/api.test.js`	Rutas HTTP sobre el doble en memoria.	`app.js`.
`tests/dominio.test.js`	Reglas de negocio.	`domain/reglas.js`.
`tests/permisos.test.js`	Cada permiso coincide con la autorización real.	`permisos.js`.
`tests/servicios.test.js`	Casos de uso sobre el doble en memoria.	`application/`.
`tests/helpers.js`	Datos y tokens de prueba.	Pruebas.
`tests/doubles/memoria.js`	Doble en memoria de los puertos.	`ports/index.js`.
`tests/soporte/flujoComun.js`	Batería común para el doble y el motor real.	Pruebas.
`tests/soporte/flujoAcceso.js`	Batería común de cuentas de acceso.	Pruebas.
`backend/db.js`	**Conexión a PostgreSQL** con `LABORATORIO_DB_URL`.	`index.js`, `scripts/`.
`backend/adapters/postgres/esquema.sql`	Esquema `laboratorio`: tablas, restricciones y permisos.	`npm run lab:esquema`.
`backend/adapters/postgres/index.js`	Adaptador: unidad de trabajo y traducción de errores.	`repositorios.js`.
`backend/adapters/postgres/repositorios.js`	Consultas SQL de cada puerto.	PostgreSQL.
`backend/application/solicitudesService.js`	Casos de uso de solicitudes de exámenes.	Puertos.
`backend/application/resultadosService.js`	Registro y validación de resultados.	Puertos.
`backend/api/routes/laboratorio.js`	Rutas de solicitudes y resultados.	Casos de uso.
`scripts/esquema.js`	Aplica el esquema; no borra datos.	`npm run lab:esquema`.
`frontend/paginas/DetalleSolicitud.jsx`	Detalle de una solicitud.	Su `api.js` y plataforma `ui`.
`tests/login.test.js`	Inicio de sesión con cuentas `lab.*`.	`accesoService.js`.
`tests/postgres.integration.test.js`	Integración con un PostgreSQL desechable.	`postgresTemporal.js`.
`tests/soporte/postgresTemporal.js`	Levanta un PostgreSQL desechable.	Pruebas de integración.

tests/ — pruebas de la plataforma común

`tests/`

Archivo	Qué hace	Se comunica con
`frontend-cliente.test.js`	Cliente HTTP y API pediátrica contra Express real.	`http.js`, `pediatria/frontend/api.js`.
`frontend-modulos.test.js`	Contrato de módulos, permisos y migas de pan.	`contrato.js`, `registro.js`.
`frontend-equipo.test.js`	Reglas de módulos pendientes y separación entre carpetas.	`instituciones/*/frontend`.
`frontend-portal.test.js`	Estados del portal; comprueba que solo navega.	`portal.js`.
`vite-proxy.test.js`	Inicio de sesión a través del proxy de Vite.	`vite.config.js`, API.
`e2e/vista-previa.e2e.js`	Recorre en navegador todas las vistas previas.	`playwright-core`, Vite.
`e2e/auditoria-ui.js`	Comprobaciones de accesibilidad y contraste.	Pruebas E2E.
`soporte/alias-frontend.mjs`	Registra los alias para pruebas en Node.	`alias-frontend-ganchos.mjs`.
`soporte/alias-frontend-ganchos.mjs`	Resuelve `@plataforma` e `@instituciones` en Node.	`apps/web/alias.js`.

12. Qué archivo tocar para cada cambio

Quiero…	Archivos a modificar
Cambiar de servidor o base MongoDB	`.env` (`MONGO_URI`, `MONGO_DB`). Nada más
Agregar un campo al paciente	`schema.js`, `validadores.js` (y aplicarlo), `pacientesService.js` (documento creado), proyecciones en `repositorios.js`, `PacienteNuevo.jsx` y `Resumen.jsx`
Agregar una ruta nueva	`api/routes/*`, el servicio en `application/`, `api.js` del módulo y la página
Agregar una operación de datos	`ports/index.js`, `repositorios.js` **y** `tests/doubles/memoria.js`; si falta alguno, falla la verificación de puertos
Agregar o cambiar una regla de negocio	`domain/reglas.js` y el servicio que la aplica
Agregar un permiso	`permisos.js`, la ruta que lo exige y `modulo.js`
Agregar una pantalla	Archivo en `paginas/` y sus entradas en `rutas` y `navegacion` de `modulo.js`
Agregar un índice	Lista `INDICES` en `adapters/mongo/index.js`
Cambiar colores	Tokens `--color-*` en `apps/web/src/styles.css`
Crear un componente común	`plataforma/ui/` y exportarlo desde `ui/index.js`, avisando al equipo
Conectar otra institución	Sección 14

13. Instalación y comandos

Se necesita Node.js 20.19 o superior y MongoDB Community 9.0. Comandos para Windows PowerShell, desde la raíz:

```
npm ci                  # dependencias del backend y las pruebas
npm run web:ci          # dependencias de la aplicación web
npm run hospital:install   # dependencias del backend del Hospital
Copy-Item .env.example .env
node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
notepad .env            # pegue el resultado en JWT_SECRET
npm run build; npm start   # aplicación completa en http://localhost:3001
```

Comando	Qué hace	Toca la base
`npm start` · `npm run dev`	Servidor en 3001 (dev se reinicia al guardar)	Sí
`npm run web:dev`	Interfaz en 5173 con vista previa	No
`npm run build`	Compila la web en `apps/web/dist`	No
`npm test`	Pruebas de plataforma y de las cuatro instituciones	Solo desechable
`npm run test:e2e`	Flujo completo en navegador	Solo desechable
`npm run test:e2e:vista-previa`	Comprueba la vista previa de desarrollo	No
`npm run db:indices`	Crea índices faltantes; no borra	Crea índices
`npm run db:validadores`	Compara documentos con los validadores	Solo lectura

Comando	Qué hace	Toca la base
`npm run db:validadores -- --aplicar`	Respalda e instala validadores	Cambia reglas
`npm run db:diagnostico`	Informe del estado de la base	Solo lectura
`npm run db:respaldo`	Respaldo en `respaldos/mongodb/<fecha>/`	Solo lectura
`npm run db:restaurar-aislado -- <carpeta>`	Restaura en un `mongod` temporal	No la original
`npm run db:crear-accesos`	Crea las cuentas iniciales y `CREDENCIALES_ACCESO.txt`	Crea cuentas
`npm run lab:esquema`	Aplica el esquema del Laboratorio; no borra datos	Crea tablas
`npm run lab:usuarios`	Crea las cuentas `lab.*`	Crea cuentas
`npm run odonto:usuarios`	Crea las cuentas `odonto.*`	Crea cuentas
`npm run hospital:usuario`	Crea una cuenta `hosp.*`	Crea cuentas

Variables de entorno

Variable	Uso	Obligatoria
`MONGO_URI`	Dirección del servidor MongoDB	No (usa localhost)
`MONGO_DB`	Nombre de la base	No
`JWT_SECRET`	Firma de los tokens; 32 caracteres o más	**Sí**
`JWT_ISSUER` · `JWT_AUDIENCE`	Emisor y audiencia del token	No
`PORT`	Puerto de la API (3001)	No
`NODE_ENV`	Entorno de ejecución	No
`CORS_ORIGIN`	Orígenes permitidos si la web está en otro dominio	No
`MONGO_ADMIN_URI`	URI con `dbAdmin` para aplicar validadores	No
`LABORATORIO_DB_URL`	Conexión a PostgreSQL del Laboratorio	Para montarlo
`ODONTOLOGIA_FIREBASE_CREDENCIALES`	Ruta a la llave de Firebase, fuera del repositorio	Para montarlo
`HOSPITAL_DB_USER` · `HOSPITAL_DB_PASSWORD`	Cuenta de MySQL del Hospital	Para montarlo
`HOSPITAL_DB_HOST` · `_PORT` · `_NAME`	Dirección y nombre de la base del Hospital	No
`MONGOD_BIN` · `MONGODB_TOOLS_BIN`	Ruta de `mongod` y de las Database Tools	No
`VITE_API_PROXY`	A dónde reenvía Vite en desarrollo	No

14. Cómo conectar otra institución

El procedimiento completo está en `docs/GUIA_EQUIPO.md`, secciones 6 y 7. En resumen:

1. Confirmar el motor de base de datos con el equipo.
2. Crear `instituciones/<x>/backend/` con dominio, casos de uso, puertos, adaptador, rutas, permisos y **su propia conexión**.
3. Crear un router `crearRouter<Institucion>` que atienda `GET /sesion`, autorice cada ruta y use el formato de errores común.
4. Exportar su composición con `router`, `prefijoUsuario` y `autenticar`, y montarla en `pediatria/backend/server.js` solo si su variable de conexión existe.
5. Completar `roles` en `modulo.js` y las operaciones en `api.js`.
6. Cambiar las páginas de vista previa por datos reales con `useCarga` y `useEnvio`.
7. Agregar una línea en `plataforma/modulos/registro.js`.
8. Agregar sus pruebas y dejar todo en verde: `npm test`, `npm run build` y las E2E.

15. Problemas frecuentes

Síntoma	Causa y solución
`Configure JWT_SECRET de al menos 32 caracteres`	Falta `.env` o conserva el valor de ejemplo
`connect ECONNREFUSED 127.0.0.1:27017`	MongoDB no está corriendo (`Get-Service MongoDB`) o `MONGO_URI` es incorrecta
`Cannot GET /` en el puerto 3001	Falta `npm run build`
El puerto está ocupado	`$env:PORT=3002` y, para Vite, `$env:VITE_API_PROXY="http://localhost:3002"`
Pruebas de MongoDB omitidas	Definir `$env:MONGOD_BIN` con la ruta de `mongod.exe`
«Institución no disponible en esta interfaz»	El backend aceptó el token, pero el módulo no está en `registro.js`
`403 INSTITUCION_NO_REGISTRADA`	Falta registrar el router en `app.js`
`401 TOKEN_INVALIDO` con un token recién emitido	`JWT_SECRET`, `JWT_ISSUER` o `JWT_AUDIENCE` distintos entre servicios
«Usuario o contraseña incorrectos»	Revise mayúsculas; los datos están en `CREDENCIALES_ACCESO.txt`. Si la base está vacía, ejecute `npm run db:crear-accesos`
Un botón visible devuelve 403	La navegación usa un permiso distinto del que exige la ruta
No aparece ninguna sección	`/api/sesion` no devuelve `permisos`

16. Observaciones del análisis

Detalles encontrados al revisar el código que conviene conocer. Ninguno impide usar el proyecto.

Pendientes conocidos

- No hay pantalla para cambiar o recuperar contraseñas ni para administrar cuentas.
- El servidor común vive dentro de la carpeta de Pediatría, aunque atiende a las cuatro instituciones.
- Si MongoDB se cae con la API en marcha, la respuesta es un `500` genérico (D-08 en la auditoría). `AUDITORIA_2026-10-31.md` detalla este y los demás defectos y su estado.

Documentación desactualizada

Dónde	Qué dice	Lo actual
`pediatria/docs/DEMOSTRACION.md`, paso 13	61 pruebas y 18 pasos E2E	El README indica los números actuales
Comentarios de `permisos.js`, `repositorios.js`, `validadores.js`	Rutas `src/api/routes`, `src/ports`, `src/schema.js`	Esas carpetas ya están en `backend/`
`docs/GUIA_EQUIPO.md` §6	El JWT se valida una vez	Se verifica dos veces: en el enrutador y en `requireAuth`. Es inofensivo
`scripts/seed.js`	El nombre sugiere datos de ejemplo	Solo crea índices
`package.json` (raíz)	Nombre `centro-pediatrico-mongodb`	El repositorio ya abarca cuatro instituciones

Límites aceptados del diseño

- La prevención de pacientes duplicados y el límite de solicitudes funcionan en memoria: valen para una sola instancia de la API.
- La detección de pacientes duplicados ignora mayúsculas, pero no espacios ni tildes.
- Las altas de catálogo en standalone no son atómicas con su auditoría.
- Un paciente no puede superar 16 MiB, límite de MongoDB por documento; el análisis en `MODELO_Y_DECISIONES.md` lo considera suficiente.

Para seguir leyendo

Documento	Para qué
`docs/GUIA_EQUIPO.md`	Contratos completos entre instituciones
`pediatria/README.md`	API detallada y pruebas
`pediatria/docs/MODELO_Y_DECISIONES.md`	Por qué MongoDB y por qué este modelo
`pediatria/docs/OPERACION_MONGODB.md`	Validadores y paso a replica set
`pediatria/docs/RESPALDO_Y_RESTAURACION.md`	Respaldar y comprobar un respaldo
`pediatria/docs/DEMOSTRACION.md`	Guion de presentación